

Nullcon HackIM 2023

Crypto - Curvy Decryptor

Nullcon HackIM 2023 Crypto - Curvy Decryptor writeup: cryptography, ECC, invalid_curve, CRT, ECDLP

EVENT	CATEGORY	TOPIC	PUBLISHED
nullcon hackim	crypto	ECC	2023-08-21

Challenge Description

Curvy Decryptor 473 points Alice has hidden 2 flags in this challenge. And even though she is willing to decrypt most ciphers, she has some basic safeguards against stealing flags. Please submit flag1 here. nc 52.59.124.14 10005

Source Files

[curvy_decryptor.py](#) ([./curvy_decryptor.py](#))
[ec.py](#) ([./ec.py](#))
[utils.py](#) ([./utils.py](#))

Source Analysis

From `curvy_decryptor.py`

PYTHON
76 lines / static highlight

```
#!/usr/bin/env python3
import os
```

```
import os
import sys
```

```
import string
from Crypto.Util import number
```

```
from Crypto.Util.number import bytes_to_long, long_to_bytes
from Crypto.Cipher import AES
```

```
from binascii import hexlify
```

```
from utils import *
from secret import flag1, flag2
```

#P-256 parameters
p =

```
0xffffffff0000000100000000000000000000000000000000ffffffffffffffffffff  
f
```

```
b = 0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
```

```
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
b
```

```
0xffffffff00000000ffffffffffffbce6faada7179e84f3b9cac2fc63255
1
```

```
G = ECPoint(curve,  
0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c29
```

0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f
5)

$$P_a = G * d_a$$

```
printable = [ord(char.encode()) for char in string.printable]
```

```

def encrypt(msg : bytes, pubkey : ECPoint):
    x = bytes_to_long(msg)
    y = modular_sqrt(x**3 + a*x + b, p)
    m = ECPoint(curve, x, y)
    d_b = number.getRandomRange(0,n)
    return (G * d_b, m + (pubkey * d_b))

def decrypt(B : ECPoint, c : ECPoint, d_a : int):
    if B.inf or c.inf: return b''
    return long_to_bytes((c - (B * d_a)).x)

def loop():
    print('I will decrypt anything as long as it does not talk
about flags.')
    balance = 1024
    while True:
        print('B:', end = '')
        sys.stdout.flush()
        B_input = sys.stdin.buffer.readline().strip().decode()
        print('c:', end = '')
        sys.stdout.flush()
        c_input = sys.stdin.buffer.readline().strip().decode()
        B = ECPoint(curve, *[int(_) for _ in B_input.split(',')])
        c = ECPoint(curve, *[int(_) for _ in c_input.split(',')])
        msg = decrypt(B, c, d_a)
        if b'EN0' in msg:
            balance = -1
        else:
            balance -= 1 + len([c for c in msg if c in
printable])
        if balance >= 0:
            print(hexlify(msg))
            print('balance left: %d' % balance)
        else:
            print('You cannot afford any more decryptions.')
            return

```

```

if __name__ == '__main__':
    print('My public key is:')
    print(P_a)
    print('Good luck decrypting this cipher.')
    B,c = encrypt(flag1, P_a)
    print(B)
    print(c)
    key = long_to_bytes((d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff))
    enc = AES.new(key, AES.MODE_ECB)
    cipher = enc.encrypt(flag2)
    print(hexlify(cipher).decode())
    try:
        loop()
    except Exception as err:
        print(repr(err))

```

Curvy Decryptor part 1

The solution of Curvy Decryptor part 1 is to find out `flag1`

Curvy Decryptor part 2

The solution of Curvy Decryptor part 1 is to find out `flag2`

The `flag2` appears to be AES encrypted with a key which is

PLAIN TEXT

1 line / static highlight

```

key = long_to_bytes((d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff))

```

So in order to recover the `flag2` we will need to break the ECC and recover `d_a`

Analysing curvy_decryptor.py

PYTHON

10 lines / static highlight

#P-256 parameters

p =

```
0xffffffff000000010000000000000000000000000000ffffffffff
f
```

$$a = -3$$
$$b =$$

0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
b

$$n =$$

```
0xffffffff00000000fffffffffffffbce6faada7179e84f3b9cac2fc63255
1
```

```
curve = EllipticCurve(p,a,b, order = n)
```

```
G = ECPoint(curve,
```

0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296,

0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5)

```
d a = bytes to long(os.urandom(32))
```

$$P_a = G * d_a$$

indeed look like a standard curve P-256 (note that $a = -3$ is equivalent to $a = p - 3$)

So there doesnt appear to be any standard weakness based on weak curve parameters.

The private key `d_a` is initialized to be per-instance system-random 128 bit number

And the Public key `P_a` is simply `G*d_a`

Encryption

PYTHON

6 lines / static highlight

```
def encrypt(msg : bytes, pubkey : ECPoint):  
    x = bytes_to_long(msg)  
    y = modular_sqrt(x**3 + a*x + b, p)  
    m = ECPoint(curve, x, y)  
    d_b = number.getRandomRange(0,n)  
    return (G * d_b, m + (pubkey * d_b))
```

The `msg` is encoded as the x coordinate of the point, the corresponding `y` is found so as to find the point on the curve to generate the point `m`

A secure random number `d_b` in range (0 - curve order) is generated as the nonce, The points `G*d_b` and `m + (pubkey * d_b)` are returned

Decryption

PYTHON

3 lines / static highlight

```
def decrypt(B : ECPoint, c : ECPoint, d_a : int):  
    if B.inf or c.inf: return b''  
    return long_to_bytes((c - (B * d_a)).x)
```

It simply reverses the encrypt function if correct `d_a` is provided
i.e if we do

PYTHON

2 lines / static highlight

```
B,c = encrypt(flag1, P_a)
xx = decrypt(B,c)
```

We will get,

$$\text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b))$$

$$= (\text{flag}_m + (P_a * d_b) - G * d_b * d_a).x$$

$$= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a).x$$

$$= (\text{flag}_m).x = \text{flag}$$

Main Loop

The main loop of the program just repeatedly asks for input of two EC points `B` and `c` and tries to decrypt it with the servers private key `d_a`

BUT

if the decryption contains `b'EN0'` i.e the start of the flag, it exits.

Otherwise it decreases the balance proportionate to the number of printable characters in the decryption.

So If we directly input the `B` and `c` corresponding to the `flag1`, it will abort and hence no flags for us :'(

But it doesnt care what points we ask it to decrypt.

So what if we try to decrypt the points `B`, `c + A`

We will get,

$$\text{decrypt}((G * d_b * d_a), \text{flag}_m + (P_a * d_b) + A)$$

$$= (\text{flag}_m + (P_a * d_b) + A - G * d_b * d_a).x$$

$$= (\text{flag}_m + A + G * d_a * d_b - G * d_b * d_a).x$$

$$= (\text{flag}_m + A).x$$

As long as we know A, we can always get the point from the x coordinate, and subtract A from it to get the original point from the curve for the sake of simplicity, we can even pick it to be G

or we can even try decrypting the points $B + A$, C which will lead to

$$\text{decrypt}((G * d_b * d_a) + A, \text{flag}_m + (P_a * d_b))$$

$$= (\text{flag}_m + (P_a * d_b) - (G * d_b + A) * d_a).x$$

$$= (\text{flag}_m + G * d_a * d_b - G * d_b * d_a - A * d_a).x$$

$$= (\text{flag}_m - A * d_a).x$$

if we choose A to be G or $-G$, we will end up with the point $\text{flag} - P_a$ or $\text{flag} + P_a$ and since we even know P_a , it will also work

With a high probability, we won't observe any `b'ENO'` in the resulting point, and if we do, we can always pick countless possibilities of `A` to make it work

PYTHON

45 lines / static highlight

```
from ec import *
from utils import *
import pwn

p = 
0xfffffffff000000010000000000000000000000000000fffffffffffbce6faada7179e84f3b9cac2fc63255
f
a = -3
b = 
0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604
n = 
0xfffffffff0000000fffffffffffffffbce6faada7179e84f3b9cac2fc63255
curve = EllipticCurve(p,a,b, order = n)
G = ECPoint(curve,
0x6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c29
6,
0x4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f
5)

HOST, PORT = "52.59.124.14", 10005
REM = pwn.remote(HOST, PORT)

REM.recvline() # My public key is:
pubkey = REM.recvline().strip()[6:-1].split(b',')
P_a = ECPoint(curve, int(pubkey[0]), int(pubkey[1]))

REM.recvline() # Good luck decrypting this cipher.
B_text = REM.recvline().strip()[6:-1].split(b',')
B = ECPoint(curve, int(B_text[0]), int(B_text[1]))
```

```

c_text = REM.recvline().strip()[6:-1].split(b',')
c = ECPoint(curve, int(c_text[0]), int(c_text[1]))

flag2_enc = bytes.fromhex(REM.recvline().strip().decode())

REM.recvline() # I will decrypt anything as long as it does not
talk about flags.

def get_decryption(B,c):
    REM.sendline("{}{}".format(B.x, B.y))
    REM.sendline("{}{}".format(c.x, c.y))
    status = REM.recvline()
    if b'cannot afford' in status:
        return -1, None
    balance = int(REM.recvline().strip().split(b': ')[-1])
    return balance, bytes.fromhex(status.strip()[6:-1].decode())

bal, BG = get_decryption(B, c+G)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p) # getting the valid y
coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-G).x, 32,'big'))
print(int.to_bytes((point_BG2-G).x, 32,'big'))

```

**Note that we only get the x coordinate of the given point
lifting the point would result in two points and we will
need to try with both of them (x,y) and (x,-y)**

The last part will also work with

PYTHON

7 lines / static highlight

```
bal, BG = get_decryption(B-G, c)
BG_int = int.from_bytes(BG)
y = modular_sqrt(BG_int**3+a*BG_int+b,p) # getting the valid y
coordinate for the x
point_BG1 = ECPoint(curve, BG_int, y)
point_BG2 = -point_BG1
print(int.to_bytes((point_BG1-P_a).x, 32, 'big'))
print(int.to_bytes((point_BG2-P_a).x, 32, 'big'))
```

And we get the flag to part 1

```
b'\x00\x00ENO{ElGam4l_1s_mult1pl1cativ3}'
```

Analysing ec.py

PYTHON

66 lines / static highlight

```
from Crypto.Util.number import inverse

class EllipticCurve(object):
    def __init__(self, p, a, b, order = None):
        self.p = p
        self.a = a
        self.b = b
        self.n = order

    def __str__(self):
        return 'y^2 = x^3 + %dx + %d modulo %d' %
(self.a, self.b, self.p)

    def __eq__(self, other):
        return (self.a, self.b, self.p) == (other.a,
other.b, other.p)
```

```

class ECPPoint(object):
    def __init__(self, curve, x, y, inf = False):
        self.x = x % curve.p
        self.y = y % curve.p
        self.curve = curve
        self.inf = inf
        if x == 0 and y == 0: self.inf = True

    def copy(self):
        return ECPPoint(self.curve, self.x, self.y)

    def __neg__(self):
        return ECPPoint(self.curve, self.x, -self.y,
self.inf)

    def __add__(self, point):
        p = self.curve.p
        if self.inf:
            return point.copy()
        if point.inf:
            return self.copy()
        if self.x == point.x and (self.y + point.y) % p
== 0:
            return ECPPoint(self.curve, 0, 0, True)
        if self.x == point.x:
            lamb = (3*self.x**2 + self.curve.a) *
inverse(2 * self.y, p) % p
        else:
            lamb = (point.y - self.y) *
inverse(point.x - self.x, p) % p
        x = (lamb**2 - self.x - point.x) % p
        y = (lamb * (self.x - x) - self.y) % p
        return ECPPoint(self.curve,x,y)

    def __sub__(self, point):
        return self + (-point)

    def __str__(self):

```

```

        if self.inf: return 'Point(inf)'
        return 'Point(%d, %d)' % (self.x, self.y)

    def __mul__(self, k):
        k = int(k)
        base = self.copy()
        res = ECPoint(self.curve, 0,0,True)
        while k > 0:
            if k & 1:
                res = res + base
            base = base + base
            k >>= 1
        return res

    def __eq__(self, point):
        return (self.inf and point.inf) or (self.x ==
point.x and self.y == point.y)

```

Looking closely at the `ECPoint` class, one would note that on the initialization of the point with arbitrary `x, y` coordinates, it works as usual and doesn't check whether the supplied `x, y` satisfy the curve equation $y^2 = x^3 + ax + b \pmod p$

This leads to an interesting vulnerability aka **Invalid Curve Attack**

Which can be noted by the facts that

1. The point addition of two points P and Q over the curve $y^2 = x^3 + ax + b \pmod p$ if $P \neq Q$ is independent of both the curve parameters a and b
2. The point doubling i.e $P = Q$ is just dependent on P and Q and a but not on b again

This means that the group addition operation is independent of the parameter b , the number of points in the group of some point P is just dependent on P and a but independent of b

So if we choose a curve C' with parameter b' and pick a valid point P' on it, and run the point addition over the original curve C , the order of point P' when used on C will be the same as order of point P' when used on C'

This implies that we are not stuck with the original prime order of P-256, but we can vary $b = -3$ such that the order of the curve C' has small factors. We can then easily find points P' with those small factors as their order by using the fact that -

If G' is the generator of C' with order $o = f_1 f_2 f_3 \dots f_n$, the point $G' * (o/f_1)$ will have the order f_1

Once we have sufficient number of small orders, we can take a chinese remainder theorem over them to recover the original private key `d_a`

To find the order of curve C' and the corresponding generators, we can utilize the greates library of sagemath

PYTHON

19 lines / static highlight

```
def get_invalid_curves(a,b,n,cutoff=10**5):
    factors, total, i = {}, 0, 0
    while total < n*2:
        i += 1
        try:
            E = EllipticCurve(GF(p), [a,i])
            order = E.order()
            n_facs = order.factor()
        except ArithmeticError: #the parameter i defines a
```

```

singular curve
        continue
    for prime, power in n_facs:
        if prime > cutoff: # dont take any factors bigger
than it
            break
        gen = E.gen(0)*(order//prime)
        factors[prime] = [int(gen[0]), int(gen[1]),i]
        total *= prime
    print(i, total)
    # key : [gen_x, gen_y, b']
    return factors

```

And we can easily bruteforce a given point

PYTHON

4 lines / static highlight

```

def bruteforce(point, generator, order):
    for i in tqdm(range(order), desc=f"bruteforcing {order=}"):
        if point.y == (generator*i).y:
            return i

```

We get the following invalid curves and we just searched over 12 values of b !

PYTHON

58 lines / static highlight

```

invalid_curves = {
3:
[7969228023927298087324538783187482347609766536506916355881757038
6218657526967,

15885657487155030912288031888128427124936813080859472376494663130
798867119982, 12],
5:
[3046358645625905271617412172472378847879731893976229152365196615

```

1233767925799,

14521026652335616630611219515390291411023400641948957680811406721
043438902186, 12],

7:

[7494160963166719022445789448670075468300539216220596044581361034
311676798234,

98511591492536111584332615786483658886888663084020823615343609813
895225923287, 0],

13:

[4423839975182234462915592734941092173433666003638590881284952749
6419061724190,

11120913773080173377402108816240868388875386584846966540375366366
3899601005809, 3],

17:

[9274144687945784364291903707116312659963917031850121885976057784
793297477861,

86301980488975426887521079169756244075123784201450393961147458432
303011672794, 10],

19:

[1086572514888378397100958947438667390524868802712580336135104196
34191398226376,

85939059343162290921933874524377315775260886906764656684571310947
58391852209, 4],

37:

[6829338390266237482283310246665103308891228336319477318479644522
260556056309,

74701668267551028200304338837410580676774474001240882947774298643
661074111881, 3],

71:

[4768989115066252021641827605080277136704470836651176690754318752
1601344242001,

76986723505258444611874235435887405018513758524921757765033540214
285112109625, 1],
97:
[1093524381327895976762698492711619330291159637003767830442148056
43475162939438,

40494199582133551395560104592591896448854412368997470369685720658
455096277720, 3],
113:
[2475842305874220823820486444323131896857191883016682295763890607
9202832915346,

64680694216633390865014362020707904150333340051904806580803858267
985332824228, 3],
179:
[8311563101649050482232865577789586416266078232566035967479206533
2260812135544,

35707141486916353816358123900356888673488260709581875628819622187
261352405569, 4],
251:
[2258959779636525724629675812850577063879996176931082468786804101
0311103597978,

22730375842099404560129412560882492093998724011749582473003682871
994848593450, 10],
389:
[6173741830680999690863059543783205227270026389202136141564002831
4169193468679,

38812622012907358702971098652989910931710935180589252493039839244
464470874161, 12],
653:
[9694668034361392030009188060702797389146046409674180327317079721
9756170839615,

82843786165425711709725308210080928839669646246950072002603300248
219042690214, 9],

823:

[1015910781698755957531099914949055069679781362409611723885092756
21325253165256,

10303457253039461887297603877250453486351456285884301634668455036
581356870325, 1],

1151:

[4942947285962241518079147671001480777229821084370946279070977308
149340420785,

16048516228456466259745940658231903287363041924322695524263376011
778714624389, 7],

1229:

[9170924868892838157497030687995914325677991135597065911779823476
1550615769703,

85856020107303390216619790339131383987393998107943546966635616939
179964220668, 1],

2447:

[1070910371096125709951362942133366829239137179860541790946439220
74841981090569,

38297847735446351346601186761335949464902974429727652825128988635
682228100545, 5],

4003:

[6963461236054763969297805073647558400000195034696325413489365933
1303767659709,

72676901546760217087111884977950279161557917843992139313558513515
10639163175, 6],

7103:

[1103235277408923562768338447688604495542910102082012557928250534
03232044044793,

34702628194678317337541067016532288256370093140368313590333192034
888987762792, 7],

7489:

[6949761078970559517405873710624251310095013019092070246743103217

2354669590563,

43000989776377667520933328800675765150040604546037676698173382008
099239610730, 1],

13003:

[6999438843130785608032257273197091727015106751101851761953056891
4812259046195,

51645889020375608054366335957352074257130320976341249224010343992
676177045239, 4],

16033:

[8015084977070128077037926080287633225724565160722043687384170856
9583336291111,

67454443034144602807761039494179913732280388550455172486599878108
995578176702, 6],

19423:

[8676331669611614620784620944308937609596654228199007187269873412
4275764832625,

14973732811888413420821129175194083916646739915937107562258163136
02284346637, 11],

30203:

[2714030676912436425321282688995125071478292918068545559928468770
2513066987645,

27465668374540052358785326933904597047991904030378513297677516281
690992773738, 1],

52183:

[2078689300620066813598051748130519896787152213077370057132725618
0224225598537,

38476450712159672989047119873673988596095516096648184067210103163
599625447149, 12],

72337:

[1686467313604327869304018557230348574367712599923341997643730247
1094264721938,

```

39982110747848740588957884598316010802865483814669576940304708444
368796141014, 9],
81173:
[4196584713467566686308967062141269929720744625991527783293989960
5119013001686,

97595102592346869875749873612676528534971198259624427507680148771
098555985918, 8]
}

```

Attack procedure

1. For each b' and G' of order o'
2. Send $B = -G'$ and $c = G'$ we will get back x coordinate of $G' * (d_a + 1)$

Note that if we send 0, it will just return b'' so no use

3. Lift the x coordinate **Over C'** to get $G' * (d_a + 1)$ But this wont help us to figure out of the two possible points. For this
4. Send another $B = -G'$ and $c = G' * 2$ to get back x coordinate of $G' * (d_a + 2)$ To figure out the correct lifting of the x coordinate to get $G' * (d_a + 1)$
5. Bruteforce and recover the ECDLP to get $d'_a = (d_a + 1) \bmod o'$
6. Combine all d'_a using chinese remainder theorem to get back original d_a
7. PROFIT ????

PYTHON

27 lines / static highlight

```
recovered_order = {}
for order, (gen_x, gen_y, b_i) in invalid_curves.items():
    gen = ECPoint(curve, gen_x, gen_y)
    bal, BG = get_decryption(-gen, gen) # gen*(da+1)
    BG_int = int.from_bytes(BG)
    y = modular_sqrt(BG_int**3+a*BG_int+b_i,p)
    point_BG = ECPoint(curve, BG_int, y)

    bal, BG2 = get_decryption(-gen, gen*2) # gen*(da+2)
    BG_int2 = int.from_bytes(BG2)
    if (point_BG+gen).x == BG_int2:
        recovered_order[order] = bruteforce(point_BG, gen, order)
    elif (point_BG-gen).x == BG_int2:
        recovered_order[order] = bruteforce(-point_BG, gen,
order)
    else:
        print("something went wrong")

mods, values = [],[]
for i,v in recovered_order.items():
    mods.append(i)
    values.append((v-1)%i)

d_a = crt(mods,values)[0]

key_int = (d_a >> (8*16)) ^ (d_a &
0xffffffffffffffffffffffffffffffff)
key = key_int.to_bytes(16, 'big')
print(AES.new(key, AES.MODE_ECB).decrypt(flag2_enc))
```

ENO{be5t_th1nk_out_0f_th3_curv3}

Solve script

[solve.py \(./solve.py\)](#)

[Suggest a correction to this article.](#)

PREVIOUS

**ACSC qualifiers 2023 Crypto -
SusCipher**

NEXT

.inputrc: VI experience in the shell